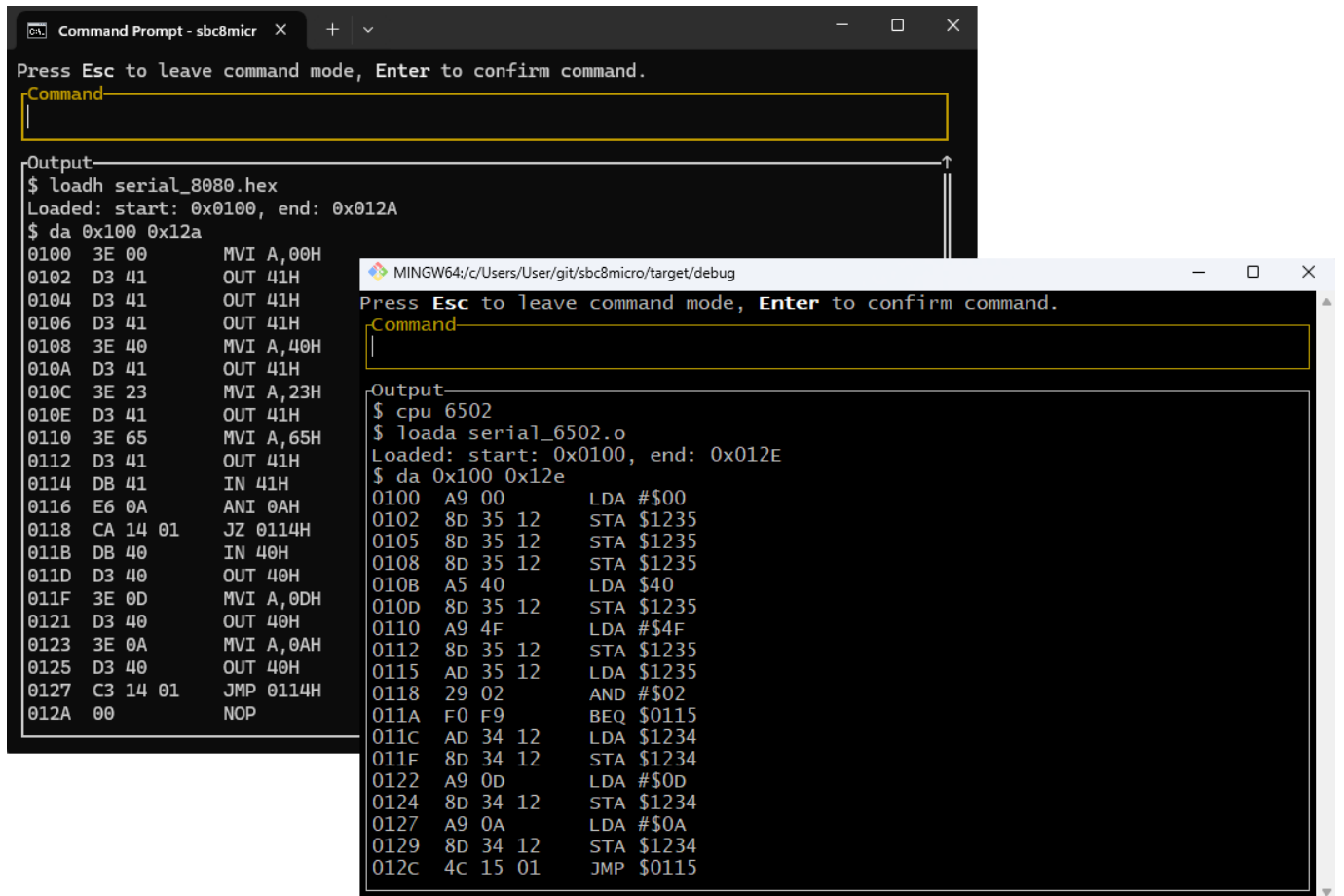


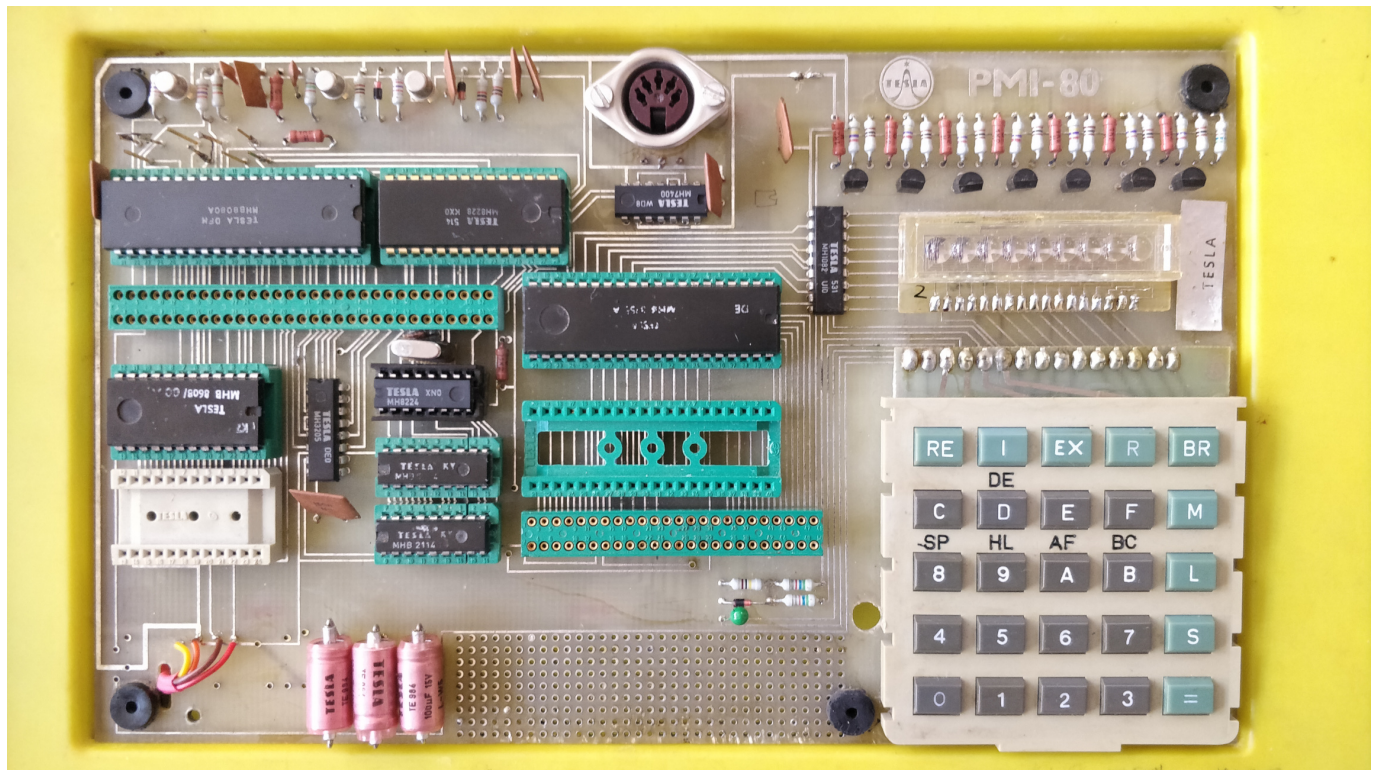


```
Command Prompt - sbc8micro
Press Esc to leave command mode, Enter to confirm command.
Command
Output
$ loadh serial_8080.hex
Loaded: start: 0x0100, end: 0x012A
$ da 0x100 0x12a
0100 3E 00 MVI A,00H
0102 D3 41 OUT 41H
0104 D3 41 OUT 41H
0106 D3 41 OUT 41H
0108 3E 40 MVI A,40H
010A D3 41 OUT 41H
010C 3E 23 MVI A,23H
010E D3 41 OUT 41H
0110 3E 65 MVI A,65H
0112 D3 41 OUT 41H
0114 DB 41 IN 41H
0116 E6 0A ANI 0AH
0118 CA 14 01 JZ 0114H
011B DB 40 IN 40H
011D D3 40 OUT 40H
011F 3E 0D MVI A,0DH
0121 D3 40 OUT 40H
0123 3E 0A MVI A,0AH
0125 D3 40 OUT 40H
0127 C3 14 01 JMP 0114H
012A 00 NOP

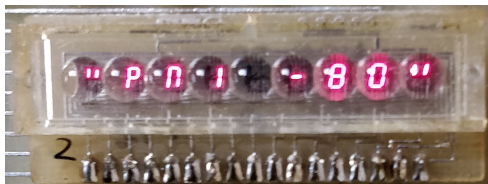
MINGW64/c/Users/User/git/sbc8micro/target/debug
Press Esc to leave command mode, Enter to confirm command.
Command
Output
$ cpu 6502
$ loada serial_6502.o
Loaded: start: 0x0100, end: 0x012E
$ da 0x100 0x12e
0100 A9 00 LDA #$00
0102 8D 35 12 STA $1235
0105 8D 35 12 STA $1235
0108 8D 35 12 STA $1235
010B A5 40 LDA $40
010D 8D 35 12 STA $1235
0110 A9 4F LDA #$4F
0112 8D 35 12 STA $1235
0115 AD 35 12 LDA $1235
0118 29 02 AND #$02
011A F0 F9 BEQ $0115
011C AD 34 12 LDA $1234
011F 8D 34 12 STA $1234
0122 A9 0D LDA #$0D
0124 8D 34 12 STA $1234
0127 A9 0A LDA #$0A
0129 8D 34 12 STA $1234
012C 4C 15 01 JMP $0115
```

## sbc8micro - 6502, 8080 emulator.

This project is a result of my enthusiasm for retro computing. It was primarily triggered when my brother gave me a single-board 8080-based microcomputer, the **PMI-80**, from 1982. At that time, it was quite a legendary board in Czechoslovakia, though it was very simple.



It has a missing 8224 clock generator, so I had to spend some time finding a new one. All the chips on that board used to be produced in Czechoslovakia, and I wanted to find the original MHB8224 produced by a company named TESLA. (It has nothing to do with Elon's Tesla; he just stole the name 😊). But my effort paid off, and the board is now fully functional:



And then a question came out: "Now what?" 😊

As it was quite inconvenient to type hexadecimal code via keyboard, I decided that the cure for this would be an EPROM emulator. The problem was that those currently available don't support the old 1kB 8708 type of ROM used on this board. But there exists the emulator [EPROM-EMU-NG: EPROM Emulator Project with Arduino](#), so I modified the software and adapted the probe so it can be used also for 1kB 8708 ROMs. My modification can be seen here: [EPROM\\_EMU\\_NG\\_FW\\_2708](#). And my enthusiasm didn't end there, and I also decided to design my own single-board microcomputer based on the Intel 8080 CPU. The design of [SBC\\_8080\\_ECB](#) has already been finished and will be used for other projects based on the ECB bus.

While working on the above mentioned projects I got an idea to write an emulator for Intel 8080 CPU so I can verify the software from convenience of my laptop. I also wanted to improve my skills related to RUST programming language so this looked like a good idea. I also asked ChatGPT about how to start the project like this one but the response I got was only related to

MOS6502 CPU. So I accepted the chalange and decided to include both 8080 and 6502 CPUs 😊. The result is a modular system that can be tailored to specific requirements and it also contains a graphical user interface so it can be used for debugging of the software, switching between CPUs, seting a breakpoints, disassembling the code, viewing an opcodes for specific CPU and many more. There is also a build in emulator of Intel 8251A serial interface which can be mapped either to IO address space (used by 8080) or CPUs memory (used by 6502). This configuration allows to connect to running "system" from a terminal like putty on Windows or Screen or Minicom on linux. Other devices can easily be added as the interface between device registers and the CPU is simple and well defined so the implementation should be straight forward.